

Design a Hit Counter

LeetCode 362 · Google · Datadog · Prometheus – ప్రతి metrics system, ప్రతి dashboard, ప్రతి "గత గంట" గ్రాఫ్ వెనక ఇదే ఉంది.

మూడు విరుపులు. మొదటిది 10,000 keys కి **447 GB** అడుగుతుంది. రెండోది – పార్శ్వపుస్తక జవాబు – **ఒకే ఒక window** కి జవాబిస్తుంది. మూడోది memory ని 6,000 రెట్లు తగ్గించి, **75% తప్పు జవాబులు** ఇస్తుంది.

ఈ doc ఎలా చదవాలి

పక్కన editor తెరిచి కలిసి రాయండి. ఇక్కడి ప్రతి output, ప్రతి శాతం నిజంగా `node` lo run చేసినదే – 1.42 కోట్ల hits తో సహా.

DEEP DIVE 03 (RATE LIMITER) చదివారా? – అయితే ఇది ఏమి కాదో ముందే చెప్తాను

Rate Limiter lo ఇప్పటికే చూశాం: fixed window సరిహద్దు దాడి, sliding window log యొక్క memory, sliding counter యొక్క ఉజ్జాయింపు, token bucket. ఆ నాలుగింటినీ ఇక్కడ మళ్ళీ చెప్పను.

Hit counter వేరే problem, మరియు తేడా ఒక్క పదంలో ఉంది: **window ఒక parameter.**

Rate limiter కి ఒకే ఒక window ఉంటుంది – "నిమిషానికి 100". దాన్ని design చేసేటప్పుడే నిర్ణయిస్తాం.

Hit counter ని ఎవరైనా ఏ window గురించైనా అడగొచ్చు – "గత 5 నిమిషాలు", "గత గంట", "గత 30 రోజులు" – మరియు అవన్నీ ఒకే structure నుంచి రావాలి. అదే §7 lo పాఠ్యపుస్తక జవాబుని విరగొడుతుంది.

విషయ సూచిక (Table of Contents)

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. అడిగింది ఏమిటి – మరియు దాగిన parameter
2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

Part 2 – మొదటి విరుపు: ప్రతి hit ని గుర్తుపెట్టుకోవడం

3. Step – ఒక జాబితా, పాతవి తీసేయడం
4. మొదటి విరుపు – 10,000 keys కి 447 GB
5. Step – వృత్తాకార buckets · పాఠ్యపుస్తక జవాబు

Part 3 – రెండో విరుపు: ఒకే ఒక window

6. Step – "ఇప్పుడు గత గంట కూడా చెప్పు"
7. రెండో విరుపు – ఆ సమాచారం structure lo లేనే లేదు
8. Step – వయసుతో resolution తగ్గడం

Part 4 – మూడో విరుపు: 6,000 రెట్లు తక్కువ memory, 75% తప్పు

9. Step – తప్పుని కొలవడం
10. మూడో విరుపు – ఒక సెకను window ని తప్పు ring కి పడేస్తుంది
11. Step – Tiers ని ఎంచుకోవడమే design

Part 5 – పూర్తి system

12. Step – తిరస్కరణ, మరియు ఈ design చేయలేనిది

13. మొత్తం code ఒకే చోట · నడిపి చూద్దాం

Part 6 – Interview lo

14. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి

15. నోటితో చెప్పాల్సిన English script

16. Follow-ups – distributed, percentiles, unique counts

17. ఏమి నేర్చుకున్నాం

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. అడిగింది ఏమిటి – మరియు దాగిన parameter

"Design a hit counter which counts the number of hits received in the past 5 minutes."

ఇది LeetCode 362, మరియు దానికి ఒక అందమైన పాఠ్యపుస్తక జవాబు ఉంది (§5). కానీ ఆ ప్రశ్నలో ఒక పదం దాగి ఉంది: "5 నిమిషాలు".

అసలు ప్రశ్న: **WINDOW** ఎవరు నిర్ణయిస్తారు?

"5 నిమిషాలు" – design సమయంలో తెలిసిందా, query సమయంలో వచ్చేదా?

Design సమయంలో తెలిస్తే

300 buckets, ఒక్కొక్కటి ఒక సెకను

$O(1)$ hit · స్థిర memory

→ §5 · 2,400 bytes · పరిష్కారమైంది

Query సమయంలో వస్తే

"గత 5 నిమిషాలు", "గత గంట",

"గత 30 రోజులు" – ఒక structure నుంచి

→ §7 · పూర్తిగా వేరే problem

మరియు నిజ ప్రపంచంలో ఎప్పుడూ రెండే

ఏ dashboard చూసినా పైన ఒక drop-down ఉంటుంది: **5m · 1h · 6h · 1d · 7d · 30d**.

అవన్నీ **ఒకే డేటా** నుంచి రావాలి – ఒక్కోదానికీ ఒక్కో counter కాదు.

మరియు ఇక్కడ ఒక కఠినమైన వ్యాపారం మొదలవుతుంది:

ఎంత వెనక్కి × ఎంత సూక్ష్మంగా × ఎన్ని keys = memory. మూడింటినీ ఒకేసారి పొందలేం.

2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

ప్రశ్న	జవాబు నా design ని ఎలా మారుస్తుంది
Window స్థిరమా, query lo వచ్చేదా?	ఇదే మొత్తం problem – §7

ప్రశ్న	జవాబు నా design ని ఎలా మారుస్తుంది
ఎంత వెనక్కి? (5 నిమి? 30 రోజులు?)	§11 – tiers ని ఇదే నిర్ణయిస్తుంది
ఎన్ని keys? (ఒక్క counter నా, URL వారీగానా?)	§4 – memory ని keys సంఖ్యతో గుణించాలి
జవాబు ఖచ్చితంగా ఉండాలా, ఉజ్జాయింపు చాలా?	అత్యంత ముఖ్యమైన ప్రశ్న – §11
సికనుకి ఎన్ని hits?	§4 – naive version యొక్క memory
Hits క్రమంలో వస్తాయా, ఆలస్యంగా వస్తాయా?	ఆలస్యం ఉంటే buckets ని వెనక్కి తాకాలి
గతం గురించి అడగొచ్చా? ("నిన్న 3 గంటలకి")	లేదు అంటే ఇది చాలా సులభం – §12

నాలుగో ప్రశ్న అడగండి – అది INTERVIEW ని తెరుస్తుంది

"జవాబు ఖచ్చితంగా ఉండాలా, లేక $\pm 2\%$ పర్సల్‌దా?"

Metrics dashboards lo జవాబు దాదాపు ఎప్పుడూ "ఉజ్జాయింపు చాలు". "గత 24 గంటల్లో 4,74,746 hits" మరియు "4,74,100 hits" – ఏ dashboard చూసేవారికీ తేడా తెలియదు.

మరియు ఆ ఒక్క అనుమతి **memory ని 500 రెట్లు తగ్గిస్తుంది** (§11).

కానీ billing కి? **ఖచ్చితత్వం తప్పనిసరి**. అందుకే ఈ ప్రశ్న అడగాలి – జవాబు మీ design ని పూర్తిగా మారుస్తుంది.

Part 2 – మొదటి విరుపు: ప్రతి hit ని గుర్తుపెట్టుకోవడం

3. Step – ఒక జాబితా, పాతవి తీసేయడం

```
class HitCounter {
  constructor(windowMs = 300_000) { this.window = windowMs; this.hits = []; }
  hit(now) { this.hits.push(now); }
  #prune(now) {
    const cutoff = now - this.window;
    let i = 0;
    while (i < this.hits.length && this.hits[i] <= cutoff) i++;
    if (i) this.hits.splice(0, i); // పాతవి తీసేయడం
    return i;
  }
  count(now) { this.#prune(now); return this.hits.length; }
}
```

సరిగ్గా పనిచేస్తుంది – మరియు ఇది ఖచ్చితమైనది, ఉజ్జాయింపు కాదు:

మొదటి hit	→ గత 5 నిమిషాల్లో: 1	
1 సె తర్వాత	→ గత 5 నిమిషాల్లో: 2	
1 నిమిష	→ గత 5 నిమిషాల్లో: 3	
5 నిమిష	→ గత 5 నిమిషాల్లో: 2	← మొదటి hit గడువు తీరింది
5 నిమిష 1 సె	→ గత 5 నిమిషాల్లో: 3	

ఎనిమిది పంక్తులు. ఇది ఖచ్చితమైన జవాబు ఇస్తుంది, ఏ window కైనా ఇవ్వగలదు (జాబితాలో అంతా ఉంది), మరియు ఏ ఉజ్జాయింపు లేదు.

మరి సమస్య ఏమిటి?

4. మొదటి విరుపు – 10,000 keys కి 447 GB

ఒక hit = ఒక timestamp = 8 bytes. అది చిన్నదే. కానీ:

సెకనుకి hits	5 నిమిషాల window	ఒక key కి memory	10,000 keys అయితే
100	30,000	0.2 MB	2.2 GB
1,000	3,00,000	2.3 MB	22.4 GB
20,000	60,00,000	45.8 MB	447.0 GB

("keys" = ఒక్కే URL, ఒక్కే user, ఒక్కే endpoint – నిజమైన systems lo వేలు)

మొదటి విరుపు – MEMORY ట్రాఫిక్ తో పాటు పెరుగుతుంది

ఒక్క counter అయితే 45.8 MB – భరించొచ్చు. కానీ **hit counter అనేది దాదాపు ఎప్పుడూ ఒక్కటి కాదు.**

మీరు కొలవాలనుకునేది "మొత్తం ఎన్ని hits" కాదు – "ఏ URL కి ఎన్ని", "ఏ user ఎన్ని", "ఏ error code ఎన్నిసార్లు". అంటే వేల counters.

మౌలిక తప్పు: జవాబు ఒక సంఖ్య. కానీ మనం ఆ సంఖ్య కోసం ప్రతి సంఘటననీ నిల్వ చేస్తున్నాం. 60 లక్షల timestamps ఉంచి, వాటిని లెక్కపెట్టి, ఒక సంఖ్య ఇస్తున్నాం.

మరియు ఆ timestamps lo ఉన్న సమాచారంలో 99.99% మనకి అవసరం లేదు – ఏ hit ఎప్పుడు వచ్చిందో కాదు, ఎన్ని వచ్చాయో మాత్రమే కావాలి.

ఒక cleanup వివరం: నేను "ఒక పెద్ద burst తర్వాత మొదటి query చాలా నెమ్మదిగా ఉంటుంది" అని ఊహించి కొలిచాను – 50 లక్షల hits తర్వాత అది **8.8 ms**. చెడ్డదే కానీ నేను ఊహించినంత కాదు, ఎందుకంటే V8 యొక్క `splice(0, i)` ఒక memmove, ఒక్కొక్క మూలకాన్నీ జరపడం కాదు.

కాబట్టి **ఆ కథని వదిలేశాను** – నిజమైన సమస్య memory, latency కాదు. (కొలిచి తప్పని తేలితే, దాన్ని రాయకూడదు.)

5. Step – వృత్తాకార buckets • పాఠ్యపుస్తక జవాబు

అవసరం లేని సమాచారాన్ని విసిరేద్దాం. ఏ hit ఎప్పుడు వచ్చిందో కాదు – **ఏ సెకనులో ఎన్ని వచ్చాయో మాత్రమే ఉంచుదాం.**

5 నిమిషాలు = **300 సెకన్లు** = 300 సంఖ్యలు. మరియు అవి **వృత్తాకారంగా** తిరుగుతాయి:

```
class BucketCounter {
  constructor(windowSec = 300) {
    this.n = windowSec;
    this.time = new Int32Array(windowSec); // ఆ bucket ఏ సెకనుది
    this.hits = new Int32Array(windowSec); // ఆ సెకనులో ఎన్ని
  }
  #slot(sec) { return ((sec % this.n) + this.n) % this.n; }

  hit(sec, n = 1) {
    const i = this.#slot(sec);
    // ఆ గడి వేరే సెకనుది అయితే – అది 300 సెకన్ల పాతది, తుడిచేయడం
    if (this.time[i] !== sec) { this.time[i] = sec; this.hits[i] = 0; }
    this.hits[i] += n;
  }
  count(sec) {
    const cutoff = sec - this.n;
    let total = 0;
    for (let i = 0; i < this.n; i++)
```

```
        if (this.time[i] > cutoff) total += this.hits[i];
        return total;
    }
}
```

ఆ `TIME[I] != SEC` తనిఖీ - ఇదే మొత్తం ఉపాయం

వృత్తాకార array lo గడి `i` ని సెకను `i`, `i+300`, `i+600` ... అన్నీ పంచుకుంటాయి. కాబట్టి ఆ గడిలో **ఏ సెకనుది** అని కూడా నిల్వ చేస్తాం.

కొత్త సెకను వచ్చినప్పుడు ఆ గడి **తనంతట తానే** తుడిచిపెట్టుకుంటుంది. **ఎప్పుడూ cleanup పని లేదు** - పాత డేటా దానంతట అదే భర్తీ అవుతుంది.

ఇది **LeetCode 362** యొక్క ఆశించిన జవాబు, మరియు ఇది నిజంగా అందమైనది.

జవాబులు 53 తో అక్షరాలా ఒకటే, మరియు:

memory: 2400 bytes - hits ఎన్ని వచ్చినా అంతే
(naive version lo అది 45.8 MB అయ్యేది)

45.8 MB → 2,400 bytes. మరియు ట్రాఫిక్ పెరిగినా అది మారదు.

మొదటి విరుపు పరిష్కారమైంది. ఇప్పుడు interviewer మాట్లాడతాడు.

Part 3 – రెండో విరుపు: ఒకే ఒక window

6. Step – "ఇప్పుడు గత గంట కూడా చెప్పు"

"బాగుంది. ఇప్పుడు dashboard lo ఒక drop-down ఉంది – 5 నిమిషాలు, 1 గంట, 1 రోజు, 30 రోజులు. అన్నిటికీ జవాబివ్వాలి."

7. రెండో విరుపు – ఆ సమాచారం structure lo లేనే లేదు

--- ఇప్పుడు interviewer: "గత గంట కూడా చెప్పు" ---
count(t) → 3 (5 నిమిషాలు మాత్రమే – ఇదే ఒక్కటి తెలుసు)
గంట? → ఈ structure lo ఆ సమాచారమే లేదు.

ఇది ఒక bug కాదు – ఆ డేటా ఉనికిలోనే లేదు. 300 సెకన్ల కంటే పాతది ఇప్పటికే తుడిచిపెట్టుకుపోయింది.

"సరే, buckets పెంచుదాం" అంటే:

5 నిమిషాలు	300 buckets ·	2 KB/key · 10,000 keys =	0.02 GB
1 గంట	3,600 buckets ·	28 KB/key · 10,000 keys =	0.27 GB
1 రోజు	86,400 buckets ·	675 KB/key · 10,000 keys =	6.44 GB
30 రోజులు	25,92,000 buckets ·	20250 KB/key · 10,000 keys =	193.12 GB

1-సెకను RESOLUTION × ఎంత వెనక్కి × ఎన్ని KEYS

Resolution స్థిరంగా ఉంచితే, memory కాలంతో పాటు నేరుగా పెరుగుతుంది

5 నిమిషాలు	0.02 GB
1 గంట	0.27 GB
1 రోజు	6.44 GB
30 రోజులు	193 GB – ఒక్క metric కోసం

మరియు ఒక dashboard lo metrics వందలు ఉంటాయి.

రెండో విరుపు - STRUCTURE ఒక్క ప్రశ్నే జవాబిస్తుంది

§5 యొక్క పరిష్కారం **అద్భుతమైనది** - కానీ అది "గత 300 సెకన్లు" అనే **ఒకే ఒక ప్రశ్నే** జవాబిస్తుంది. Window అనేది structure lo **కాల్చి పెట్టబడింది**.

మరియు దాన్ని సాగదీయడం పనిచేయదు: 30 రోజులకి **25 లక్షల buckets** - అందులో దాదాపు అన్నీ ఖాళీ లేదా ఎవరూ అడగని వివరాలు.

మౌలిక తప్పు: మనం **ప్రతి సెకనున ఒకేలా విలువైనదిగా** చూస్తున్నాం. కానీ అది నిజం కాదు -

ఇప్పుడే జరిగిన సెకను: దాని వివరం ముఖ్యం. ("ఇప్పుడు స్పైక్ వచ్చిందా?")

మూడు వారాల క్రితం సెకను: ఎవరికీ ఆ సెకను అక్కరలేదు. ఆ రోజు ఎంత ట్రాఫిక్ వచ్చిందో చాలు.

8. Step - వయసుతో resolution తగ్గడం

ఆ గమనిక పరిష్కారం: **పాత డేటాని ముతకగా ఉంచడం.**

నాలుగు RINGS - ఒక్కొక్కటి ఒక స్థాయి సూక్ష్మత

ఇటీవలిది సూక్ష్మంగా - పాతది ముతకగా

1 సెకను × 300 గత 5 నిమిషాలు	10 సెకన్లు × 360 గత 1 గంట	60 సెకన్లు × 1440 గత 1 రోజు	900 సెకన్లు × 2880 గత 30 రోజులు
--------------------------------	------------------------------	--------------------------------	------------------------------------

ప్రతి hit **నాలుగు rings** ని పెంచుతుంది. Query ఏ window అడిగినా -

దాన్ని కప్పగలిగే **అతి సూక్ష్మమైన ring** ఎంచుకుని, అందులోని buckets కలుపుతాం.

మొత్తం buckets: $300 + 360 + 1440 + 2880 = 4,980$

1-సెకను resolution తో 30 రోజులు: **25,92,000 buckets**

అంటే **520 రెట్లు తక్కువ** - మరియు అదే తగ్గింపుకి ఒక ధర ఉంది (§9).

ఆ ధరని **కొలవకుండా** ఈ design ని ప్రతిపాదించకూడదు.

```
class Ring {
  constructor(stepSec, buckets) {
    this.step = stepSec; this.n = buckets;
    this.key = new Int32Array(buckets).fill(-1); // ఆ bucket ఏ కాలఖండానిది
    this.hits = new Float64Array(buckets);
  }
  get spanSec() { return this.step * this.n; }
  #k(sec) { return Math.floor(sec / this.step); }

  add(sec, n) {
    const k = this.#k(sec), i = ((k % this.n) + this.n) % this.n;
    // ఆ గడి వేరే కాలఖండానిది అయితే - అది పాతది, తుడిచేయడం
    if (this.key[i] !== k) { this.key[i] = k; this.hits[i] = 0; }
    this.hits[i] += n;
  }

  // (now - window, now] పరిధిని తాకే buckets మొత్తం
  sum(now, windowSec) {
    const newest = this.#k(now), oldest = this.#k(now - windowSec + 1);
    let total = 0;
    for (let i = oldest; i < newest; i++) {
```

```
const k = this.key[i];  
if (k >= oldest && k <= newest) total += this.hits[i];  
}  
return total;  
}  
}
```

ఇది §5 యొక్క అదే ఉపాయం – **కేవలం step అనే ఒక parameter చేరింది.** step = 1 అయితే అది సరిగ్గా §5. మరియు ఇప్పుడు నాలుగు rings ని పక్కపక్కన పెట్టొచ్చు.

Part 4 – మూడో విరుపు: 6,000 రెట్లు తక్కువ memory, 75% తప్పు

9. Step – తప్పుని కొలవడం

ఇక్కడ ఒక నియమం: ఒక ఉజ్జాయింపు design ప్రతిపాదిస్తే, దాని తప్పుని కొలవాలి. "ఉజ్జాయింపు చాలు" అనేది ఒక సాకు కాదు – అది ఒక సంఖ్యతో రుజువు కావాలి.

కాబట్టి: 30 రోజుల నిజమైన-ఆకారపు ట్రాఫిక్ (పగలు ఎక్కువ, రాత్రి తక్కువ), ఒక ఖచ్చితమైన reference (ప్రతి hit నిల్వ), మరియు 200 యాదృచ్ఛిక window పొడవులు:

```
class Exact { // నెమ్మదైనది, కానీ ఖచ్చితమైనది
  constructor(){ this.h = []; }
  hit(s, n=1){ for (let i=0;i<n;i++) this.h.push(s); }
  count(now, w){ const c = now - w; let t = 0;
    for (const x of this.h) if (x > c && x <= now) t++; return t; }
}
```

మొదటి tiers – సహజమైన ఎంపిక: 1s, 1 నిమిషం, 1 గంట, 1 రోజు.

10. మూడో విరుపు – ఒక సెకను window ని తప్పు ring కి పడేస్తుంది

30 రోజుల ట్రాఫిక్ · 1,42,54,500 hits · ప్రతి window కి 200 యాదృచ్ఛిక పొడవులు

window పరిధి	ముతక tiers (4.9 KB)	
	సగటు	గరిష్టం
1-5 నిమిషాలు	0.00%	0.00%
5-60 నిమిషాలు	2.10%	14.31%
1-24 గంటలు	10.02%	75.45%
1-30 రోజులు	5.06%	69.38%

మూడో విరుపు - 75% తప్పు అనేది ఒక DASHBOARD కాదు, ఒక అబద్ధం

Memory 30 MB నుంచి **4.9 KB** కి దిగింది - 6,000 రెట్లు. కానీ "గత 12 గంటల్లో ఎన్ని hits?" అనే ప్రశ్నకి జవాబు **75% తప్పుగా** రావచ్చు.

కారణం ఒక సెకను. "1 గంట" window కి 60-సెకన్ల ring సరిపోతుంది (అది సరిగ్గా 3600s కప్పుతుంది). కానీ window **3601 సెకన్లు** అయితే? - ఆ ring సరిపోదు, కాబట్టి తర్వాతి ring కి పడుతుంది: **1-గంట resolution.**

అంటే ఒక సెకను తేడా వల్ల resolution **60 రెట్లు ముతకగా** అయింది, మరియు సరిహద్దు వద్ద ఒక పూర్తి గంట డేటా లెక్కలోకి వచ్చేసింది - ఒక గంట window మీద అది దాదాపు 100%.

మౌలిక తప్పు: Tiers మధ్య **ఖాళీలు** ఉన్నాయి. ప్రతి ring సరిగ్గా తర్వాతి ring మొదలయ్యే చోట ఆగిపోతోంది, కాబట్టి ఆ అంచు వద్ద ఉన్న ప్రతి query **కొండ దిగి పడుతుంది.**

11. Step - Tiers ని ఎంచుకోవడమే design

పరిష్కారం: **rings ఒకదాన్నొకటి ఉదారంగా కప్పేలా** ఎంచుకోవడం. ప్రతి ring తన resolution ని **చాలా దూరం వరకు** మోయాలి, తర్వాతి దాని అంచు వరకే కాదు.

	ముతక tiers	సన్నని tiers
ring 0	1s × 300 = 5 నిమి	1s × 300 = 5 నిమి
ring 1	60s × 60 = 1 గంట	10s × 360 = 1 గంట
ring 2	3600s × 24 = 1 రోజు	60s × 1440 = 1 రోజు
ring 3	86400s × 30 = 30 రోజులు	900s × 2880 = 30 రోజులు

కప్పే కాలం ఒకటే. మారినది **resolution** - ప్రతి ring 6-60 రెట్లు సూక్ష్మంగా ఉంది, కాబట్టి buckets ఎక్కువ.

window పరిధి	ముతక tiers (4.9 KB)		సన్నని tiers (58.4 KB)	
	సగటు	గరిష్టం	సగటు	గరిష్టం
1-5 నిమిషాలు	0.00%	0.00%	0.00%	0.00%
5-60 నిమిషాలు	2.10%	14.31%	0.35%	2.71%
1-24 గంటలు	10.02%	75.45%	0.15%	1.40%
1-30 రోజులు	5.06%	69.38%	0.06%	0.59%

memory: ముతక 4.9 KB · సన్నని 58.4 KB
1-సెకను resolution తో 30 రోజులు: 30 MB
→ సన్నని tiers 520 రెట్లు తక్కువ

12 రెట్లు ఎక్కువ MEMORY • 50 రెట్లు తక్కువ తప్పు

4.9 KB → 58.4 KB (12× ఎక్కువ), మరియు గరిష్ట తప్పు 75.45% → 1.40% (54× తక్కువ).

మరియు 58.4 KB ఇప్పటికీ పూర్తి 1-సెకను resolution (30 MB) కంటే 520 రెట్లు తక్కువ.

ఇదే ఈ problem యొక్క నిజమైన design నిర్ణయం. "Rings వాడతాను" అనేది ఒక ఆలోచన; ఏ rings అనేది ఒక design – మరియు ఆ ఎంపికని కొలవకుండా చేయలేరు.

Interview lo చెప్పాల్సిన వాక్యం: "The tier layout is the design, and I'd pick it by measuring the error against an exact reference, not by intuition."

ఒక పద్ధతి పొరపాటు, నేను చేసినది: మొదటి కొలతలో నేను now ని గత రోజులో యాదృచ్ఛికంగా ఎంచుకున్నాను – మరియు తప్పు 100% వచ్చింది. కారణం: rings వృత్తాకారమైనవి, కాబట్టి "నిన్నటి 3 గంటలకి గత 5 నిమిషాలు ఎన్ని?" అని అడగలేం – ఆ డేటా ఎప్పుడో భర్తీ అయిపోయింది.

అది ఒక bug కాదు, ఈ design యొక్క ఒక లక్షణం (§12). కానీ నా పరీక్ష దాన్ని ఒక తప్పుగా కొలిచింది. సరైన కొలత: now ని స్థిరంగా ఉంచి, window పొడవుని మార్చడం.

Part 5 – పూర్తి system

12. Step – తిరస్కరణ, మరియు ఈ design చేయలేనిది

ఒక మంచి design తాను ఏమి చేయలేదో స్పష్టంగా చెప్పాలి:

```
count(now, windowSec) {  
  if (windowSec < 1) return { ok: false, reason: `BAD_WINDOW: ${windowSec}` };  
  const r = this.rings.find(x => x.spanSec >= windowSec);  
  if (!r)  
    return { ok: false,  
            reason: `WINDOW_TOO_LONG: ${windowSec}s > ${this.maxWindow}s` };  
  return { ok: true, count: r.sum(now, windowSec), resolution: r.step,  
          maxErrorPct: +(r.step / windowSec * 100).toFixed(2) };  
}
```

జవాబుతో పాటు దాని నాణ్యత కూడా ఇవ్వడం

`count` ఒక సంఖ్య మాత్రమే ఇవ్వట్లేదు – అది `resolution` మరియు `maxErrorPct` కూడా ఇస్తోంది.

ఎందుకు ముఖ్యం? – ఒక dashboard "4,74,746" అని చూపిస్తే, చూసేవారు దాన్ని ఖచ్చితమైనదని అనుకుంటారు. కానీ అది $\pm 0.07\%$ ఉజ్జాయింపు.

ఒక ఉజ్జాయింపు జవాబుని, అది ఉజ్జాయింపు అని చెప్పకుండా ఇవ్వడం – అదే నిజమైన bug. Chart కి "~4.75 లక్షలు" అని చూపించడానికీ, alert rule కి "ఈ సంఖ్య $\pm 1\%$ లోపు" అని తెలియడానికీ – ఆ metadata కావాలి.

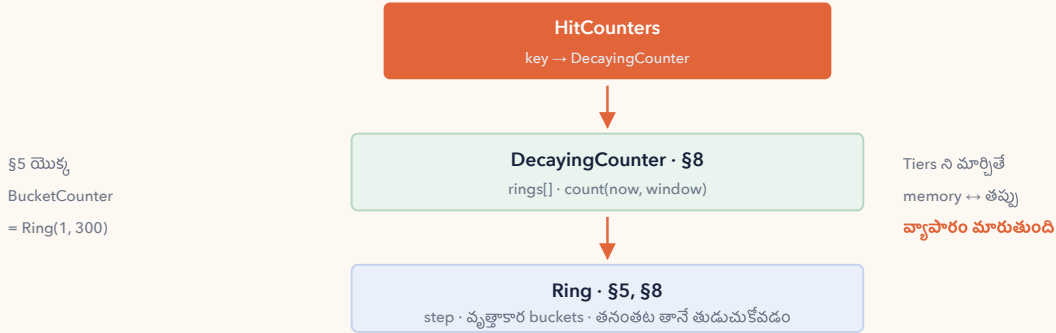
మరియు ఈ design చేయలేనివి – మూడు

- 1 • గతం గురించి అడగలేం.** "నిన్న మధ్యాహ్నం 3 గంటలకి గత 5 నిమిషాల్లో ఎన్ని?" – ఆ డేటా ఎప్పుడో భర్తీ అయింది. Rings ఎప్పుడూ "ఇప్పటి నుంచి వెనక్కి" మాత్రమే జవాబిస్తాయి.
- 2 • ఆలస్యంగా వచ్చిన hits.** ఒక hit 10 నిమిషాలు ఆలస్యంగా వస్తే, ring 0 (5 నిమిషాలు) దాన్ని స్వీకరించదు – ఆ bucket ఇప్పటికే భర్తీ అయింది. ముతక rings దాన్ని పట్టుకుంటాయి.
- 3 • గరిష్ట window ని దాటలేం.** 30 రోజులకి మించి అడిగితే `WINDOW_TOO_LONG` – మరియు అది సరైన ప్రవర్తన. తెలియని దానికి ఒక సంఖ్య ఇవ్వడం కంటే, తెలియదని చెప్పడం మేలు.

13. మొత్తం code ఒకే చోట · నడిపి చూద్దాం

నిర్మాణం · ఒకే ఉపాయం, మూడు స్థాయిల్లో

Ring ఒక్కటే ఆలోచన - మిగతావన్నీ దాని పాతలు



```
'use strict';
class Ring {
  constructor(stepSec, buckets) {
    if (stepSec < 1 || buckets < 1)
      throw new Error(`BAD_RING: ${stepSec}x${buckets}`);
    this.step = stepSec; this.n = buckets;
    this.key = new Int32Array(buckets).fill(-1); // ఆ bucket ఏ కాలఖండానిది
    this.hits = new Float64Array(buckets);
  }
  get spanSec() { return this.step * this.n; }
  get bytes() { return this.n * 12; } // Int32 + Float64
  #k(sec) { return Math.floor(sec / this.step); }

  add(sec, n) {
    const k = this.#k(sec), i = ((k % this.n) + this.n) % this.n;
    if (this.key[i] !== k) { this.key[i] = k; this.hits[i] = 0; }
    this.hits[i] += n;
  }
  sum(now, windowSec) {
    const newest = this.#k(now), oldest = this.#k(now - windowSec + 1);
    let total = 0;
    for (let i = 0; i < this.n; i++) {
      const k = this.key[i];
      if (k >= oldest && k <= newest) total += this.hits[i];
    }
    return total;
  }
}

const DEFAULT_TIERS = [[1, 300], [10, 360], [60, 1440], [900, 2880]];

class DecayingCounter {
  constructor(tiers = DEFAULT_TIERS) {
    this.rings = tiers.map(([step, n]) => new Ring(step, n));
    // సన్నని నుంచి ముతక వరకు క్రమంలో ఉండాలి
    for (let i = 1; i < this.rings.length; i++)
      if (this.rings[i].step <= this.rings[i-1].step ||
          this.rings[i].spanSec <= this.rings[i-1].spanSec)
        throw new Error('TIERS_NOT_INCREASING');
  }
  hit(sec, n = 1) { for (const r of this.rings) r.add(sec, n); }
```

```

count(now, windowSec) {
  if (windowSec < 1) return { ok: false, reason: `BAD_WINDOW: ${windowSec}` };
  const r = this.rings.find(x => x.spanSec >= windowSec);
  if (!r)
    return { ok: false,
      reason: `WINDOW_TOO_LONG: ${windowSec}s > ${this.maxWindow}s` };
  return { ok: true, count: r.sum(now, windowSec), resolution: r.step,
    maxErrorPct: +(r.step / windowSec * 100).toFixed(2) };
}

get maxWindow() { return this.rings[this.rings.length - 1].spanSec; }
get bytes() { return this.rings.reduce((s, r) => s + r.bytes, 0); }
describe() {
  return this.rings.map(r => ({ step: r.step, buckets: r.n, covers: r.spanSec }));
}
}

class HitCounters {
  #map = new Map();
  constructor(tiers = DEFAULT_TIERS) { this.tiers = tiers; }
  #get(key) {
    if (!this.#map.has(key)) this.#map.set(key, new DecayingCounter(this.tiers));
    return this.#map.get(key);
  }
  hit(key, sec, n = 1) { this.#get(key).hit(sec, n); }
  count(key, now, windowSec) {
    const c = this.#map.get(key);
    return c ? c.count(now, windowSec)
      : { ok: true, count: 0, resolution: this.tiers[0][0], maxErrorPct: 0 };
  }
  get keys() { return [...this.#map.keys()]; }
  get bytes() {
    let t = 0; for (const c of this.#map.values()) t += c.bytes; return t;
  }
}

```

--- Tiers ---

1s × 300 = 300s (0.1 గంటలు)
 10s × 360 = 3600s (1.0 గంటలు)
 60s × 1440 = 86400s (24.0 గంటలు)
 900s × 2880 = 2592000s (720.0 గంటలు)
 మొత్తం 58.4 KB · గరిష్ట window 30 రోజులు

--- ప్రాథమిక ప్రవర్తన ---

మొదటి hit → గత 5 నిమిషాల్లో: 1
 1 సె → గత 5 నిమిషాల్లో: 2
 1 నిమి → గత 5 నిమిషాల్లో: 3
 5 నిమి → గత 5 నిమిషాల్లో: 2
 5 నిమి 1 సె → గత 5 నిమిషాల్లో: 3

--- ఒకే structure, పలు windows ---

1 నిమిషం → 2 (resolution 1s · గరిష్ట తేడా ≤1.67%)
 5 నిమిషాలు → 3 (resolution 1s · గరిష్ట తేడా ≤0.33%)
 1 గంట → 5 (resolution 10s · గరిష్ట తేడా ≤0.28%)
 1 రోజు → 5 (resolution 60s · గరిష్ట తేడా ≤0.07%)
 30 రోజులు → 5 (resolution 900s · గరిష్ట తేడా ≤0.03%)
 60 రోజులు → WINDOW_TOO_LONG: 5184000s > 2592000s

```

--- పలు keys ---
/home      → {"ok":true,"count":8,"resolution":1,"maxErrorPct":0.33}
/search    → {"ok":true,"count":12,"resolution":1,"maxErrorPct":0.33}
/unknown   → {"ok":true,"count":0,"resolution":1,"maxErrorPct":0}
2 keys · 116.7 KB

--- తిరస్కరణలు ---
window చాలా పెద్దది : { ok: false, reason: 'WINDOW_TOO_LONG: 5184000s > 2592000s' }
window సున్నా       : { ok: false, reason: 'BAD_WINDOW: 0' }
tiers క్షమంలో లేవు  : TIERS_NOT_INCREASING

```

ఈ output ని పంక్తి పంక్తిగా చదువుదాం

"ప్రాథమిక ప్రవర్తన" §3 మరియు §5 తో అక్షరాలా ఒకటే – 1, 2, 3, 2, 3. మూడు వేర్వేరు designs, ఒకే జవాబులు. అదే §11 యొక్క పట్టిక నిరూపిస్తున్నది.

అరు windows, ఒకే structure – 1 నిమిషం నుంచి 30 రోజుల వరకు. §7 lo ఇది అసాధ్యం.

resolution పెరుగుతూ, maxErrorPct తగ్గుతూ – ఇది విరుద్ధంగా అనిపిస్తుంది కానీ సరైనదే: 30 రోజుల window కి 900-సెకన్ల buckets వాడుతున్నాం, కానీ 900 సెకన్లు అనేది 30 రోజుల్లో **0.03%** మాత్రమే. **Window పెద్దదయ్యేకొద్దీ ముతక resolution ఎక్కువ క్షమార్హం.**

60 రోజులు → WINDOW_TOO_LONG – తెలియనిదానికి ఒక సంఖ్య ఇవ్వకుండా, తెలియదని చెప్పింది.

2 keys · 116.7 KB – ఒక్కో key కి 58.4 KB. §4 lo అది ఒక్కో key కి 45.8 MB.

దశల నుంచి ఇక్కడికి – ఏమి చేరింది

ఎక్కడ నుంచి	ఏమి చేరింది	ఎందుకు
§3	Timestamps జాబితా	మౌలిక అస్థిపంజరం, ఖచ్చితమైనది
§4 (విరుపు)	Buckets	10,000 keys కి 447 GB
§5	<code>key[i] !== k</code> తనిఖీ	Cleanup అనేదే అవసరం లేదు
§7 (విరుపు)	<code>Ring + step</code>	Structure ఒక్క window కే జవాబిచ్చింది
§8	నాలుగు rings	పాత సెకన్ల వివరం ఎవరికీ అక్కరలేదు
§10 (విరుపు)	ఉదారంగా కప్పే tiers	ఒక సెకను తేడా → 75% తప్పు
§11	కొలిచిన తప్పు పట్టిక	ఉజ్జాయింపుని కొలవకుండా ప్రతిపాదించకూడదు
§12	<code>resolution</code> , <code>maxErrorPct</code>	ఉజ్జాయింపుని ఉజ్జాయింపు అని చెప్పాలి

Part 6 – Interview Io

14. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి

45 నిమిషాల TIME BUDGET

పాఠ్యపుస్తక జవాబుని వేగంగా ఇవ్వు, దాని హద్దుని మరే చూపించండి

6 నిమిష

Clarify – window స్థిరమా? ఉజ్జాయింపు చాలా? ఎన్ని keys?

9 నిమిష

జాబితా → buckets · **LeetCode 362**

11 నిమిష – Rings

వయసుతో resolution తగ్గడం

12 నిమిష – తప్పు

tiers ఎంపిక · ఇక్కడ మరు నిలబడతారు

7 నిమిష

distributed · percentiles · unique

ఏమి తప్పక చెప్పాలి

1. **"Window స్థిరమా, query io వచ్చేదా?" అని మొదట అడగండి** (§1) – ఇది రెండు పూర్తిగా వేర్వేరు problems.
2. **వృత్తాకార bucket ఉపాయం** (§5) – `time[i] != sec` వల్ల cleanup అవసరం లేదు. ఇది 90 సెకన్లు, మరియు ఇది LeetCode 362 యొక్క ఆశించిన జవాబు.
3. **దాని హద్దుని మరే చూపించండి** (§7) – "this answers exactly one window; a dashboard needs six." Interviewer అడిగేదాకా ఆగొద్దు.
4. **Resolution వయసుతో తగ్గడం** (§8) – మరియు "RRDtool, Graphite, Prometheus అన్నీ ఇదే చేస్తాయి" అని పేరు పెట్టండి.
5. **Tiers ఎంపికని కొలవాలి** (§11) – "I'd pick the tiers by measuring error against an exact reference." ఇదే ఈ interview io మిమ్మల్ని వేరు చేసేది.

ఏమి వదిలేయాలి

- **HitCounters (multi-key wrapper) రాయొద్దు** – "wrap it in a map keyed by metric name" ఒక వాక్యం.
- **Ring.sum ని optimise చేయొద్దు** – ఇది buckets అన్నిటినీ scan చేస్తుంది; ఒక ring io అవి కొన్ని వేలే. ప్రస్తావించి ముందుకీ.
- **Distributed** – అడిగితేనే (§16).
- **తప్పు సంఖ్యలని గుర్తుపెట్టుకోవద్దు** – "I'd measure it" అనేది సంఖ్య కంటే విలువైనది.

15. నీటితో చెప్పాల్సిన English script

"First question, and it changes everything: is the window fixed at five minutes, or does the caller choose it? And second – does the answer have to be exact, or is a couple of percent acceptable?

If the window is fixed, there's a clean answer. Don't store timestamps – store counts per second in a circular array of three hundred slots, along with which second each slot belongs to. When a hit arrives and the slot holds a different second, that slot is at least five minutes stale, so you zero it and start over. There's no cleanup pass at all; old data overwrites itself. That's constant memory regardless of traffic – about two and a half kilobytes instead of forty-six megabytes at twenty thousand hits per second.

That memory point matters more than it looks, because a hit counter is almost never one counter. You want hits per URL, per user, per status code – thousands of keys. At twenty thousand hits a second, storing raw timestamps for ten thousand keys is about four hundred and fifty gigabytes.

But I'd flag the limitation immediately rather than waiting to be asked: that structure answers exactly one window. Every real dashboard has a drop-down – five minutes, an hour, a day, thirty days – and they all have to come from the same data. Widening the array doesn't work: thirty days at one-second resolution is two and a half million buckets per key, which is a hundred and ninety gigabytes across ten thousand keys.

The insight is that not every second is equally valuable. For the last few minutes I want per-second detail, because that's where you see a spike. For three weeks ago, nobody cares about a particular second – the daily total is plenty. So I'd keep several rings at different resolutions: one-second buckets covering the recent window, ten-second buckets covering an hour, one-minute buckets covering a day, fifteen-minute buckets covering a month. Every hit increments all of them, and a query picks the finest ring that spans the requested window. That's what RRDtool and Graphite and Prometheus all do.

Now the part I'd want to be judged on. Choosing those tiers is the actual design decision, and it can't be done by intuition. My first tier layout looked natural – one second, one minute, one hour, one day – and I measured it against an exact reference over thirty days of traffic. Average error was fine, but the worst case was seventy-five percent, because a window one second longer than a ring's span falls all the way to the next ring, which might be sixty times coarser. Making the rings overlap generously instead – ten seconds covering an hour rather than sixty – cost twelve times the memory, fifty-eight kilobytes instead of five, and cut the worst-case error from seventy-five percent to one point four.

Last thing: I'd return the resolution and the error bound alongside the count. An approximate answer presented as an exact one is the real bug – the chart should say about four hundred and seventy-five thousand, and an alert rule should know its own tolerance."

16. Follow-ups – distributed, percentiles, unique counts

Follow-up	జవాబు	మారే classes
"Hits ఆలస్యంగా వస్తే?"	<code>add(sec, n)</code> ఇప్పటికే గత సెకను తీసుకుంటుంది – కానీ ఆ bucket భర్తీ అయితే పోతుంది	0
"కొన్ని hits 5 కి బదులు 100 విలువ"	<code>hit(key, sec, 100)</code> ఇప్పటికే ఉంది	0
"ఒక్కో window కి వేరే tiers"	<code>HitCounters(tiers)</code> ఇప్పటికే parameter	0
"Graph కావాలి, ఒక సంఖ్య కాదు"	<code>Ring</code> యొక్క buckets అదే graph – వాటిని తిరిగి ఇవ్వడం	+1 method
"సగటు కాదు, p99 latency కావాలి"	కింద చూడండి	వేరే structure
"ఎంత మంది ప్రత్యేక users?"	కింద చూడండి	వేరే structure
"పది servers, ఉమ్మడి జవాబు"	కింద చూడండి	Aggregation layer

Distributed – మరియు ఇక్కడ ఒక అందమైన లక్షణం

"పది servers ఒక్కొక్కటి తన సొంత rings ఉంచుకుంటే – ఉమ్మడి జవాబు కావాలంటే ఏమి చేయాలి?"

కేవలం కూడాలి. Rings అనేవి గణనలు, మరియు గణనలు **కలపగలిగేవి** (mergeable). రెండు servers యొక్క ఒకే కాలఖండపు buckets ని కూడితే సరైన మొత్తం వస్తుంది.

ఇది యాదృచ్ఛికం కాదు – **అదే కారణంగా metrics systems ఇలా design చేయబడ్డాయి.** ఒక server 58 KB పంపితే, మొత్తం క్లస్టర్ యొక్క 30 రోజుల చరిత్ర వస్తుంది.

Naive version (§3) ఇలా కలపలేదు – timestamps జాబితాలని కలపాలంటే వాటిని క్రమబద్ధీకరించాలి, మరియు అది megabytes పంపడం."

Percentiles మరియు unique counts

"ఇవి రెండూ ఈ structure తో సాధ్యం కావు, మరియు కారణం ఒకటే: అవి కలపగలిగినవి కావు.

- **p99 latency** – రెండు servers యొక్క p99 లని కూడితే క్లస్టర్ p99 రాదు. దీనికి **t-digest** లేదా **HDR histogram** కావాలి – అవి కలపగలిగే ఉజ్జాయింపు నిర్మాణాలు.
- **ప్రత్యేక users** – రెండు గణనలని కూడితే నకిలీలు రెండుసార్లు లెక్కవుతాయి. దీనికి **HyperLogLog** – అది ~12 KB lo కోట్ల ప్రత్యేక విలువలని ~2% తప్పుతో లెక్కిస్తుంది.

మరియు మూడింటిలోనూ ఒకే నమూనా: ఖచ్చితత్వాన్ని కొంచెం వదిలేసి, **స్థిరమైన memory** మరియు **కలపగలిగే తనం** పొందడం. ఈ doc lo చేసినది సరిగ్గా అదే – HyperLogLog మరియు t-digest ఈ ఆలోచన యొక్క మరింత తెలివైన రూపాలు."

17. ఏమి నేర్చుకున్నాం

ఆలోచన	ఇక్కడ ఎలా కనిపించింది	ఇంకొక్కడ వస్తుంది
జవాబు ఒక సంఖ్య అయితే, సంఘటనలు నిల్వ చేయొద్దు	\$5 buckets	Aggregates, counters, gauges
తనంతట తానే తుడుచుకునే నిల్వ	<code>key[i] !== k</code> (§5)	Ring buffers, TTL caches
పాత డేటా తక్కువ విలువైనది	Resolution decay (§8)	RRDtool, Prometheus, log retention
ఉజ్జాయింపుని కొలవాలి	75% vs 1.4% (§11)	Deep Dive 03 §8, 05 §10
Parameter ఎంపికే design	Tiers (§11)	Cache size, batch size, timeouts
ఉజ్జాయింపుని ఉజ్జాయింపు అని చెప్పాలి	<code>maxErrorPct</code> (§12)	Dashboards, APIs, ML outputs
కలపగలిగే తనం ఒక లక్షణం	Distributed (§16)	HyperLogLog, t-digest, CRDTs

ఒక చివరి ఆలోచన – ఈ **DOC LO** నేను రెండు కథలని వదిలేశాను

ఒకటి: "ఒక పెద్ద burst తర్వాత మొదటి query చాలా నెమ్మది" అని ఊహించి కొలిచాను. 50 లక్షల hits కి అది **8.8 ms** – నేను ఊహించినంత నాటకీయం కాదు. కాబట్టి ఆ కథని వదిలేశాను (§4 note).

రెండోది: తప్పుని కొలిచేటప్పుడు మొదట **100% తప్పు** వచ్చింది. కారణం నా కొలత తప్పు – నేను గతం గురించి అడుగుతున్నాను, మరియు rings అది చేయలేవు (§10 note).

రెండూ ఒకే పాఠం: ఒక కొలత మీ కథని సమర్థించకపోతే, రెండు అవకాశాలే ఉన్నాయి – **కథ తప్పు, లేదా కొలత తప్పు**. రెండింటినీ పరిశీలించాలి, మరియు ఏది తప్పునా దాన్ని రాయకూడదు.

"నేను ఊహించినది నిజం కాలేదు" అనేది ఒక వైఫల్యం కాదు – **అదే కొలవడం యొక్క ఉద్దేశం**.

ఇక్కడి నుంచి ఎక్కడికి

ఈ series lo ఇప్పటివరకు: **01 Parking Lot · 02 Cache · 03 Rate Limiter · 04 BookMyShow · 05 Splitwise · 06 Elevator · 07 Pub-Sub · 08 HashMap · 09 Chess · 10 Meeting Scheduler · 11 Food Delivery · 12 File System · 13 Leaderboard · 14 Text Editor · 15 Library · 16 Tic-Tac-Toe · 17 Autocomplete · 18 Snake & Ladder · 19 Hit Counter.**

Deep Dive 03 (Rate Limiter) దీనికి జంట – అక్కడ window ఒకటే మరియు జవాబు ఒక నిర్ణయం; ఇక్కడ window ఒక parameter మరియు జవాబు ఒక సంఖ్య.

Hit Counter – అడుగు అడుగునా · ఈ doc lo ఉన్న ప్రతి output, ప్రతి శాతం నిజంగా `node` lo run చేసి తీసినదే 